

Deliverable 4: Technical Report

Capability Composition Model and Vector Space Framework

Computer Science Systems Group

October 4, 2026

1 Problem Definition

The objective is to formulate a vector embedding framework for applications defined within a formal system of states, capabilities, and goals. The goal is mapping complex operational workflows into a unified, computable vector space that enables formal compositional reasoning.

2 Design Requirements

The design demands representing capabilities as dense vectors encoding functional semantics and operational attributes. The critical requirement is preserving compositionality within a unified vector space, ensuring compounded capabilities retain exact mathematical constraints.

3 Related Embedding Approaches

While traditional vector space models evaluate text or semantic proximity, this framework targets procedural state transitions and operational compounding. Prior approaches lack combined operational attribute tracking (cost, reliability, availability) with state preconditions and effects.

4 Proposed Representation

Capabilities are embedded into a unified $2d + 3$ dimensional vector space. This structure allocates d dimensions for preconditions, d dimensions for effects, and 3 dimensions for operational attributes: cost (c), reliability (r), and availability (a).

5 Mathematical Formulation

For a capability C , the vector is structured as:

$$\mathbf{v}_C = [\vec{p}^T, \vec{e}^T, c, r, a]^T \in \mathbb{R}^{2d+3} \quad (1)$$

6 Capability Composition Model

Capability compatibility is evaluated via the inner product of effects and preconditions:

$$\text{Compat}(C_1, C_2) = \langle \vec{e}_1, \vec{p}_2 \rangle \quad (2)$$

Composition $C_{12} = C_2 \circ C_1$ follows:

$$\begin{aligned}\vec{p}_{12} &= \max(\vec{p}_1, \vec{p}_2 - \vec{e}_1) \\ \vec{e}_{12} &= \max(\vec{e}_1, \vec{e}_2) \\ c_{12} &= c_1 + c_2, \quad r_{12} = r_1 \cdot r_2, \quad a_{12} = \min(a_1, a_2)\end{aligned}$$

7 Implementation

The core framework, compatibility metrics, and composition logic are implemented in C++17.

8 Experimental Methodology

Experiments were executed to validate: 1. Compatibility metric evaluation across capability pairs. 2. Recursive composition operator correctness. 3. Cosine similarity distinction across implementation variants (API vs. DB). 4. Filtering of irrelevant capabilities.

9 Results

Table 1: Experimental Results Summary

Experiment	Metric	Observed Output
Compatibility Test	Inner Product	1.0 (Exact Match)
Composition Math	State Vectors	Exact theoretical match
Implementation Variant	Cosine Similarity	0.89 (High Semantic Affinity)
Irrelevant Filtering	Similarity Score	0.00 (Filtered cleanly)

10 Analysis

The unified vector approach preserves functional semantics while maintaining computational efficiency. Using inner products for compatibility and exact operators for composition allows fast operational chaining.

11 Limitations

The framework relies on a predefined state variable vocabulary. Furthermore, scaling state dimension d increases memory usage during large-scale composition graph traversals.

12 Conclusion

The capability composition model provides a mathematically rigorous framework for mapping, validating, and chaining operational capabilities in a unified dense vector space.